

Python

```
>>> TOTAL = 0
>>> def addToTotal(amount):
...     global TOTAL
...     TOTAL += amount
...     return TOTAL
...
>>> addToTotal(10)
10
>>> addToTotal(10)
20
>>> TOTAL
20
```

The `addToTotal()` function does have a side effect, because it modifies an element that exists outside of the function: the `TOTAL` global variable.

Side effects can be more than mere changes to global variables. They include updating or deleting files, printing text onscreen, opening a database connection, authenticating to a server, or any other manipulation of data outside of the function. Any trace that a function call leaves behind after returning is a side effect.

If a function is deterministic and has no side effects, it's known as a *pure function*. Only pure functions should be memoized. You'll see why in the next sections when we memoize the recursive Fibonacci function and the impure functions of the `doNotMemoize` program.

Memoizing the Recursive Fibonacci Algorithm

Let's memoize our recursive Fibonacci function from Chapter 2. Remember that this function is extraordinarily inefficient: on my computer, the recursive `fibonacci(40)` call takes 57.8 seconds to compute. Meanwhile, an iterative version of `fibonacci(40)` is literally too fast for my code profiler to measure: 0.000 seconds.

Memoization can greatly speed up the recursive version of the function. For example, [Figure 7-2](#) shows the number of function calls the original and memoized `fibonacci()` functions make for the first 20 Fibonacci numbers. The original, non-memoized function is doing an extraordinary amount of unnecessary computation.